

ARES 코드 수정 보고서

BLE 명령 파이프라인 최적화

(주)코리아사이언스 / 동명대학교 게임학부

2026년 5월 19일

1 개요

ARES 로버의 블록 코딩 사용 중 "LED 1번 켜기 → LED 2번 켜기"와 같이 연속된 명령 사이에 체감 가능한 지연이 발생하던 문제를 분석하여, 명령당 누적 약 250ms에 달하던 지연을 출력 명령 기준 약 40ms 수준으로 줄였다. 변경 대상은 Web/commandexecutor.js와 Pico/main.py이며, 명령의 의미에 따라 응답 라운드트립을 선택적으로 생략하는 정책을 양쪽에 일관되게 도입하였다.

2 문제 진단

단일 LED 명령 한 번이 종단 간에 소요하는 시간 예산은 다음과 같이 구성되어 있었다.

#	위치	원인	시간
1	commandexecutor.js:199	sendData(cmd, true)로 모든 명령에 응답 대기	라운드트립
2	bluetooth.js:396	CHUNK_DELAY=50ms (constants.js:14)	50ms
3	main.py:86	RECEIVE_TIMEOUT_MS=50ms 무조건 대기	50ms
4	commandexecutor.js:213	모든 명령 후 setTimeout(100)	100ms
5	BLE GATT	connection interval (HM-10 기본)	30--70ms

표 1: 단일 LED 명령의 누적 지연(개선 전) — 합계 약 250ms.

정상 동작이라도 누적 지연이 약 250ms / 명령에 달했다. 카운트 다운, 파티 조명처럼 짧은 간격의 LED 시퀀스가 “끊기는” 체감의 주된 원인이다.

3 변경 정책 — 명령별 분리

명령을 두 부류로 나누어 응답 대기 여부와 cooldown을 다르게 적용한다.

부류	명령	응답 대기
Fire-and-forget	LED_ON, LED_OFF, MAIN_LED_*, [v0 v1 v2 v3 v4] (LED 패턴), MSG, CLEAR_DISPLAY, SERVO_FORWARD/BACKWARD/LEFT/RIGHT/STOP (연 속), DC_FORWARD/BACKWARD/STOP (연속), GUN_FIRE	아니오
응답 대기 유지	BUZZER_ON,F,D (D초 blocking), SERVO_t*/DC_t* (N초 blocking), SLEEP,N (N초 blocking), DISTANCE, MAGNET, PING	예

표 2: 명령 부류와 응답 대기 정책. Pico에서 blocking으로 처리되거나 값을 반환하는 명령은 응답 대기를 유지하여 타이밍과 값 정합성을 보존한다.

4 Web 측 변경 — Web/commandexecutor.js

4.1 핵심 코드

```
// 즉시 완료 명령 - Pico가 blocking 처리하지 않으므로 응답 대기 불필요.
// 시간 의존적(BUZZER_ON, SERVO_t*, DC_t*, SLEEP) 또는 값 반환(DISTANCE,
// MAGNET, PING)은 이 집합에 넣지 말 것. 응답 대기로 두어야 타이밍/값이
// 보장된다.
FIRE_AND_FORGET_HEADS: new Set([
  'LED_ON', 'LED_OFF',
  'MAIN_LED_ON', 'MAIN_LED_OFF',
  'MSG', 'CLEAR_DISPLAY',
  'SERVO_FORWARD', 'SERVO_BACKWARD', 'SERVO_LEFT', 'SERVO_RIGHT', 'SERVO_STOP',
  'DC_FORWARD', 'DC_BACKWARD', 'DC_STOP',
  'GUN_FIRE',
]),
```

```

_isFireAndForget(command) {
  if (command.startsWith '[')) return true;    // LED 패턴 [v0 v1 v2 v3 v4]
  const head = command.split(',')[0];
  return this.FIRE_AND_FORGET_HEADS.has(head);
},

async sendCommand(command) {
  if (!state.isExecuting) {
    Logger.add('[중단] 실행이 중단되었습니다', 'warning');
    return;
  }
  BluetoothManager.updateStatus('명령 실행 중...', STATUS_COLORS.ORANGE);

  const fireAndForget = this._isFireAndForget(command);

  try {
    await BluetoothManager.sendData(command, !fireAndForget);
    if (DEBUG) Logger.add(`[완료] ${command}`, 'info');
  } catch (error) {
    // ... 에러 처리 ...
  }

  // 송신 간격 가드.
  // - fire-and-forget: UART(9600 baud)/BLE 버퍼 오버플로우 방지 최소 마진.
  // - 응답 대기: 이미 라운드트립을 거쳤으므로 가드 단축.
  const cooldown = fireAndForget ? 40 : 20;
  await new Promise(resolve => setTimeout(resolve, cooldown));
}

```

4.2 효과

- 출력 명령(LED, SERVO 연속, DC 연속 등): BLE 라운드트립과 100ms cooldown이 모두 사라짐 → 명령당 약 40ms.
- 응답 대기 명령: cooldown만 100ms→20ms 단축. 의미적 동기화는 그대로 유지.

5 Pico 측 변경 — Pico/main.py

웹의 분류와 동일 기준으로 Pico에서도 응답 송신 여부를 결정한다. 또한 단일 chunk 명령에 대한 무조건 50ms 대기를 제거한다.

5.1 NO_RESPONSE_CMDS 및 _needs_response

```
class AresRover:
    # 응답 송신 생략 명령 (Web/commandexecutor.js의 FIRE_AND_FORGET_HEADS와 동기화).
    # 즉시 완료되는 출력 명령은 응답 라운드트립을 생략하여 처리량을 높이고,
    # 응답 매칭 어긋남(LED_ON 응답이 다음 명령 슬롯에 잘못 들어가는 현상)을 차단한다.
    NO_RESPONSE_CMDS = (
        "LED_ON", "LED_OFF", "MAIN_LED_ON", "MAIN_LED_OFF",
        "MSG", "CLEAR_DISPLAY",
        "SERVO_FORWARD", "SERVO_BACKWARD", "SERVO_LEFT", "SERVO_RIGHT", "SERVO_STOP",
        "DC_FORWARD", "DC_BACKWARD", "DC_STOP",
        "GUN_FIRE",
    )

    def _needs_response(self, data):
        """응답 송신 여부. fire-and-forget 명령은 False (NO_RESPONSE_CMDS 참조)."""
        if data.startswith("["):
            # LED 패턴 [v0 v1 v2 v3 v4]
            return False
        head = data.split(", ", 1)[0]
        return head not in self.NO_RESPONSE_CMDS
```

5.2 _read_uart_line() 폴링 방식 교체

기존 구현은 매 호출마다 `utime.sleep_ms(RECEIVE_TIMEOUT_MS)`로 50ms를 무조건 대기한 뒤 두 번째 read를 수행했다. newline이 이미 도착한 단일 chunk 명령에도 적용되어 처리량 천장이 약 16--17 cmd/s에 묶여 있었다. 폴링으로 교체하여 newline 발견 즉시 종료하도록 한다.

```
def _read_uart_line(self):
    """UART에서 완전한 라인 읽기.
    newline 발견 즉시 종료 - 단일 chunk 명령에서 무조건 50ms를 기다리지 않는다.
    멀티 chunk 명령(LED 패턴, SYS_SET 등)은 RECEIVE_TIMEOUT_MS까지 폴링하여 안전.
    """
    if not self.uart.any() and not self.rx_buffer:
        return None

    start = utime.ticks_ms()
```

```

while utime.ticks_diff(utime.ticks_ms(), start) < RECEIVE_TIMEOUT_MS:
    if self.uart.any():
        chunk = self.uart.read()
        if chunk:
            self.rx_buffer += chunk.decode('utf-8', 'ignore')

        # 버퍼 오버플로우 방지
        if len(self.rx_buffer) > MAX_BUFFER_SIZE:
            print("[UART] 버퍼 오버플로우, 초기화")
            self.rx_buffer = ""
            return None

        # newline 즉시 종료
        if '\n' in self.rx_buffer:
            newline_idx = self.rx_buffer.index('\n')
            complete_line = self.rx_buffer[:newline_idx].strip()
            self.rx_buffer = self.rx_buffer[newline_idx + 1:]
            return complete_line
        else:
            utime.sleep_ms(2)

# 타임아웃: 라인 종료자 없는 부분 데이터 반환
if self.rx_buffer:
    data = self.rx_buffer.strip()
    if data and not data.endswith(','):
        self.rx_buffer = ""
        return data

return None

```

5.3 run()의 응답 조건부 송신

```

# 변경 전
else:
    response = self._process_command(data)
    self._send_response(response)

# 변경 후
else:
    response = self._process_command(data)
    if self._needs_response(data):
        self._send_response(response)

```

6 기대 효과 (정량)

명령 종류	변경 전	변경 후
LED_ON 등 fire-and-forget	약 250ms	약 40ms
BUZZER_ON,F,D 등 blocking	$D + 250\text{ms}$	$D + 20\text{ms}$
DISTANCE/MAGNET 등 값 반환	약 250ms	RTT + 20ms
Pico 처리량(단일 chunk)	$\approx 16\text{--}17 \text{ cmd/s}$	$\approx 100^+ \text{ cmd/s}$

표 3: 명령 부류별 종단 간 지연 비교 (단위: ms).

추가로 응답 매칭 어긋남(이전 fire-and-forget 명령의 응답이 다음 응답 대기 명령의 pendingResolve 슬롯에 잘못 들어가는 코너 케이스) 위험이 구조적으로 제거된다. Pico에서 응답 자체를 보내지 않으므로 침범할 응답이 존재하지 않는다.

7 잔여 위험 및 후속 작업

변경된 두 파일은 같이 갱신되어야 한다. 한쪽만 갱신되면:

- Web만 갱신: 출력 명령에 응답 대기를 안 하지만 Pico는 여전히 응답을 보냄 → BLE notify 누적, 응답 매칭 어긋남 위험 잔존.
- Pico만 갱신: Web이 출력 명령에 응답 대기를 하지만 Pico가 응답을 안 보냄 → 매 명령 5초 timeout.

7.1 남은 병목 — UART baud rate

가장 본질적인 천장은 BLE 모듈 ↔ Pico 사이의 9600 baud UART이다. HM-10/BT05의 AT 명령(AT+BAUD8 등)으로 38400 또는 115200 baud로 상향하고 Pico/main.py의 UART_BAUDRATE를 동기화하면 byte당 전송 시간을 4--12배 단축할 수 있다. BLE 모듈 펌웨어 설정과 Pico 코드 동시 수정이 필요하므로 별도 작업으로 분리한다.

7.2 권장 회귀 테스트

1. 카운트다운 — LED_OFF 5개 연속 소등. 명령 사이 체감 지연이 거의 사라져야 한다.
2. LED 패턴 + 거리 센서 혼합 — [v0 v1 v2 v3 v4] 직후 DISTANCE. 거리값이 1 같은 LED 응답으로 오염되지 않는지 확인.
3. 부저 사이렌 — BUZZER_ON,400,0.5 와 BUZZER_ON,800,0.5의 반복. 타이밍 정확성이 유지되어야 한다.
4. 시간 제한 모터 — SERVO_tFORWARD,N, DC_tFORWARD,N의 N초 동기화가 그대로 유지되는지 확인.

8 변경 파일 요약

파일	변경 요지
Web/commandexecutor.js	FIRE_AND_FORGET_HEADS 집합 도입, sendCommand()에서 분기 처리, cooldown 분리(40 / 20ms).
Pico/main.py	NO_RESPONSE_CMDS 클래스 변수, _read_uart_line() 폴링 방식으로 교체, _needs_response() 도입, run()에서 조건부 _send_response().

표 4: 본 보고서가 다루는 코드 수정 범위.